Lesson 2: コントローラ入力

Controller Input

StampFly Workshop

今日のゴール / Today's Goal

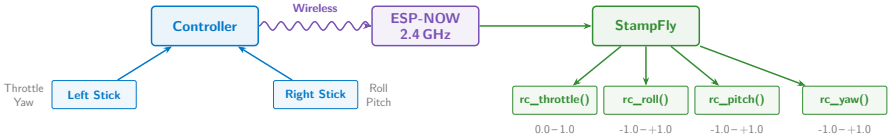
目標

スティック値を読み取り、モータをオープンループで操作する

- ESP-NOW 無線通信の仕組みを理解
- スティック値の正規化（ADC → 浮動小数点）
- オープンループ制御の限界を体感

ESP-NOW 通信フロー / Communication Flow

Controller → ESP-NOW → StampFly API



コントローラ API

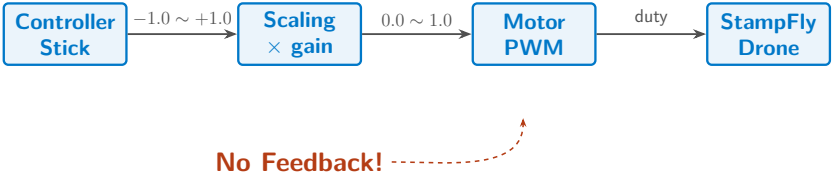
関数	説明	値域
`rc_throttle()`	スロットル	0.0 – 1.0
`rc_roll()`	ロール	-1.0 – +1.0
`rc_pitch()`	ピッチ	-1.0 – +1.0
`rc_yaw()`	ヨー	-1.0 – +1.0
`is_armed()`	ARM 状態	true / false

ペアリング手順

StampFly のボタンを **3 秒長押し** → コントローラと自動ペアリング

オープンループ制御 / Open-Loop Control

Open-Loop Control



問題点

フィードバックがないため、外乱（風・重心ずれ）に対応できない！
→ Lesson 5–6 で PID 制御を導入して解決

実習: スロットル → モータ / Hands-on

```cpp
#include "workshop_api.hpp"
static uint32_t tick = 0;

void setup() {
    ws::print("Lesson 2: Controller Input");
}
void loop_400Hz(float dt) {
    tick++;
    float throttle = ws::rc_throttle();
    // Direct throttle-to-motor (open loop)
    ws::motor_set_all(throttle);
    // Print every 200ms
    if (tick % 80 == 0) {
        ws::print("T=%.2f R=%.2f P=%.2f Y=%.2f",
            throttle, ws::rc_roll(),
            ws::rc_pitch(), ws::rc_yaw());
    }
}
```

student.cpp

チェックポイント / Checkpoint

確認事項

- ☐ コントローラとペアリングできた
- ☐ ARM 後、スロットルでモータが回る
- ☐ シリアルモニタに T/R/P/Y 値が表示される

次のレッスン

Lesson 3: LED 制御 — システム状態を LED で可視化